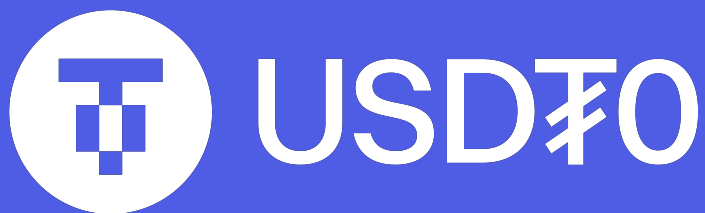


Everdawn USDT0 Stellar Deployment Assessment



August 7, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
Overview	5
Approach	6
Deployment Verification	6
Reproducible Build (SEP-58)	6
Source-Archive Integrity	7
On-Chain WASM Hash and Build Metadata	7
USDT0 SAC (network built-in)	8
Ownership Chain	8
OneSig Signer Set (point-in-time snapshot)	9
Asset-Issuer Lock	9
LayerZero Pathway Wiring	10
Trust Assumptions and Privileged Roles	14
Conclusion	15
Appendix	16
Issue Classification	16

Summary

Type	Stablecoins	Total Issues	0 (0 resolved)
Timeline	From 2026-07-29 To 2026-07-29	Critical Severity Issues	0 (0 resolved)
Languages	Rust	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	0 (0 resolved)
		Notes & Additional Information	0 (0 resolved)

Scope

This report provides a deployment verification of the USDT0 contracts deployed on the Stellar network (Pubnet/mainnet).

The deployment was verified against the **published source archives** distributed with the deployment (the SEP-58 verifiable-build archives for [everdawn-oft](#), [everdawn-sac-manager](#), and [onesig](#)). These archives are the source of truth: each one's SHA-256 is recorded as the [source_sha256](#) inside the corresponding on-chain binary, and rebuilding from the archive reproduces that binary byte-for-byte. The OFT and SAC Manager archive source trees byte-match the [Everdawn-Labs/usdt0-oft-contracts stellar-contracts main](#) branch ([ef3db2a](#) at the time of review); the OneSig archive source tree byte-matches [LayerZero-Labs/monorepo-external](#) at commit [0781f4e](#) (tag 0.2.111). The archives are repackaged build bundles rather than [git archive](#) snapshots, so these commit references are informational; the verification itself anchors on the archive hashes, not on any repository state.

The three archives ([everdawn-oft-contracts-source.zip](#), [everdawn-sac-manager-contracts-source.zip](#), and [onesig-contracts-source.zip](#)) were obtained from the [verification gist](#) published by LayerZero and shared by the client (pinned at revision [3d17dd24...](#)), which also contains the verification playbook. Each archive's SHA-256 is checked against the [source_sha256](#) baked into the corresponding on-chain binary.

Following the initial delivery of this report, the client additionally asked to verify the LayerZero configuration of the Stellar OFT. The remote-side wiring calldata for all 21 [<chain> ↔ Stellar](#) pathways (6 transactions per chain) was shared as a second gist, the [wiring gist](#), pinned at revision [cd8abce2d689ae48147f900fd4c65c2795e04af5](#). The corresponding checks are recorded in the "LayerZero Pathway Wiring" section below.

The verification covered four contracts:

Contract	Address
USDT0 OFT	CB0W0LFSDM5PZXNFIVDMP5NZ7U2GSIHED6H6R446Q0HF266XINKUMME6

Contract	Address
USDT0 SAC Manager	CA3GUWLOS3QKN6WNRAELSUDSKLDTVTWEDJ3KLGAJG3SIWGA5L3KZYWGJ
OneSig	CBCZ5CETG3XR5MZVDC7QBD0TIH6P7MOLUH2SSC52J3NVBYIV45D4QKR6
USDT0 SAC	CBSJZEI05C7KC2SF3MKSNNXXJSW5G3VTNBX4ATMKUI3B2MR4JKM4R26YF

The USDT0 asset issuer account is

[GATISXX6BZ6NC7IKQBY37CJD4S0ZL3CYZJWXEDG6JVIY4WBS6KXJHN6Q](#).

Overview

USDT0 is an Omnichain Fungible Token (OFT) built on LayerZero, giving USDT native cross-chain transfer without third-party bridges. On Stellar the deployment is made of a Soroban OFT contract, a Stellar Asset Contract (SAC) that represents the classic [USDT0](#) asset, a SAC Manager that holds the SAC admin (mint/burn authority), and a OneSig threshold multisig that owns both the OFT and the SAC Manager.

Authority flows in a chain: the SAC's [admin](#) is the SAC Manager, and the SAC Manager and OFT are both owned by the OneSig multisig. Once the asset issuer is locked, the OneSig multisig is the root owner of both contracts, holding role-assignment and upgrade authority over them.

The three custom contracts (OFT, SAC Manager, OneSig) are Soroban WASM produced by [SEP-58](#) verifiable builds, so each binary records the container image, build options, and the SHA-256 of the source archive it was built from. The USDT0 SAC is implemented by the network itself and has no uploaded WASM; it is verified by re-deriving its deterministic address from the asset and network passphrase.

Approach

The deployment verification consisted of the following checks:

1. Reproducible-build (SEP-58) equivalence: rebuild each custom WASM from source and compare byte-for-byte with the on-chain binary.
2. Source-archive integrity: confirm each source archive's SHA-256 matches the `source_sha256` recorded in the on-chain build metadata.
3. On-chain WASM hash and build-metadata checks for the three custom contracts.
4. SAC identity: re-derive the SAC address from the asset and confirm it is a network built-in asset contract.
5. Ownership-chain verification: SAC admin, SAC Manager owner, and OFT owner.
6. OneSig signer-set snapshot: threshold, signer set, and executor requirement.
7. Asset-issuer lock: confirm the issuer can no longer sign, so it can neither mint `USDT0` directly nor reclaim the SAC admin.
8. LayerZero pathway wiring (added on client request): verify the remote-side wiring calldata for all 21 pathways against the intended configuration, cross-check it against current on-chain state on each remote chain, and read the Stellar-side peer state via Soroban RPC.

Deployment Verification

The tables below record each check with its expected value and its result (✓ `Confirmed`, ⚠, or ✗). The exact commands are in the LayerZero verification notes for the deployment checks, and in the accompanying evidence scripts for the pathway-wiring checks.

Reproducible Build (SEP-58)

Contract	Package	Expected  <code>Verified</code>	Result
USDT0 OFT	<code>everdawn-of</code>	<code>71356 bytes, sha256=c1e1fce3259166b6bc17ff00 85319bb7a312cb4d4ab1dded98e60ae 32a43a7ca</code>	✓ <code>Confirmed</code>

Contract	Package	Expected  Verified	Result
USDT0 SAC Manager	everdawn-sac-manager	22041 bytes, sha256=82aebc69ff8398deaf4ce031bb7624af949ade992c4cbe890655735cda0d2cda	✓ Confirmed
OneSig	onesig	28784 bytes, sha256=7f59b1e8865ffd1bfb3437707ac5b12d87a3a828827e200e3ac9418809195945	✓ Confirmed

Source-Archive Integrity

Source archive	Expected SHA-256 (= on-chain <code>source_sha256</code>)	Result
everdawn-ofc-contracts-source.zip	6498f563d4afe26ef012993c2fdd1674e07085c56fc81415145f7ea1411f8141	✓ Confirmed
everdawn-sac-manager-contracts-source.zip	68f5fa971f4d8240e34542ec5df1bd820454ce501cb4ae270612fab8a336c61f	✓ Confirmed
onesig-contracts-source.zip	5accc7b7ab205f1cabb776c649378481728b513f47d36f4b130ba5ad385160d9	✓ Confirmed

On-Chain WASM Hash and Build Metadata

Contract	Expected on-chain WASM hash	Expected <code>source_sha256</code> (meta)	Result
USDT0 OFT	c1e1fce3259166b6bc17ff0085319bb7a312cb4d4ab1dded98e60ae32a43a7ca	6498f563...f8141	✓ Confirmed

Contract	Expected on-chain WASM hash	Expected <code>source_sha256</code> (meta)	Result
USDT0 SAC Manager	<code>82aebc69ff8398deaf4ce031bb7624af949ade992c4cbe890655735cda0d2cda</code>	<code>68f5fa97...c61f</code>	✓ Confirmed
OneSig	<code>7f59b1e8865ffd1bf3437707ac5b12d87a3a828827e200e3ac9418809195945</code>	<code>5accc7b7...160d9</code>	✓ Confirmed

All three record the same build image ([docker.io/stellar/stellar-cli@sha256:c32774bf...](https://hub.docker.com/r/stellar/stellar-cli)), Rust `1.90.0`, and Soroban SDK `25.1.1`.

USDT0 SAC (network built-in)

Check	Expected	Result
Re-derived SAC address (<code>contract id asset</code>)	<code>CBSJZEI05C7KC2SF3MKSXJJSW5G3VTNBX4ATMKUI3B2MR4JKM4R26YF</code>	✓ Confirmed
SAC is a network built-in asset contract	<code>info hash</code> returns the built-in-asset error	✓ Confirmed
Token <code>name</code> / <code>symbol</code> / <code>decimals</code> (+ <code>admin</code>)	<code>USDT0:GATISXX6...</code> / <code>USDT0</code> / <code>7</code> (admin = SAC Manager <code>CA3GUWLO...</code>)	✓ Confirmed

Ownership Chain

Check	Expected	Result
USDT0 SAC <code>admin</code>	SAC Manager <code>CA3GUWLO...YWGJ</code>	✓ Confirmed
USDT0 SAC Manager <code>owner</code>	OneSig <code>CBCZ5CET...QKR6</code>	✓ Confirmed
USDT0 OFT <code>owner</code>	OneSig <code>CBCZ5CET...QKR6</code>	✓ Confirmed

OneSig Signer Set (point-in-time snapshot)

Field	Expected	Result
<code>version</code>	<code>0.0.1</code>	✓ Confirmed
<code>threshold</code>	<code>3</code>	✓ Confirmed
<code>total_signers</code>	<code>5</code>	✓ Confirmed
<code>get_signers</code>	<code>00f6d2b4...</code> , <code>4192d72c...</code> , <code>488a3d37...</code> , <code>5f0dca10...</code> , <code>7d8e979d...</code>	✓ Confirmed
<code>executor_required</code>	<code>false</code>	✓ Confirmed
<code>onesig_id</code>	<code>30600</code>	✓ Confirmed

Owner-level actions on the OFT and SAC Manager, including role assignment and upgrades, require 3 of these 5 secp256k1 signers.

Asset-Issuer Lock

Check	Expected	Result
Issuer thresholds (low/med/high)	all <code>0</code>	✓ Confirmed
Issuer signers	single key, weight <code>0</code> (locked)	✓ Confirmed
Operation order	<code>set_admin</code> before the lock, then <code>set_options</code> (master weight → 0)	✓ Confirmed

The SAC-admin handover appears in the issuer's operation history as an `invoke_host_function` (Horizon reports the Soroban `set_admin` call under that operation type, not as a native `set_admin`); it is dated `2026-07-26 20:46:59Z`, immediately before the `set_options` lock at `2026-07-26 20:50:21Z`. The handover-then-lock order means the issuer signed the admin transfer while it still could, then permanently disabled its own signing.

Because the master key now has weight `0` with no other positive-weight signer, the issuer can neither mint `USDT0` directly nor reclaim the SAC admin. Mint and burn authority therefore

roots in the SAC admin (the SAC Manager) and its OneSig owner. Both contracts also expose granular RBAC roles.

LayerZero Pathway Wiring

The client shared the remote-side wiring calldata for every `<chain> ↔ Stellar` pathway (21 chains, 6 transactions per chain). At the time of this review (2026-08-04) those remote-side transactions had **not yet been executed**: the checks below therefore verify the *proposed* calldata against the intended configuration and against current on-chain state, not an active pathway. The Stellar-side wiring transactions are proposed separately through OneSig and were not part of the shared material; the Stellar side was instead verified by reading the deployed state directly (Soroban read-only simulations against the OFT and the LayerZero endpoint). MP1 and IOTA L1 are marked as non-wired in the client's configuration and have no wiring transactions.

Pathway-invariant checks (all 21 chains)

Check	Expected	Result
<code>SetPeer</code> peer value	Stellar OFT as bytes32: <code>0x5d672cb21b3afcdda54546c7f5b9fd346920e41f8fe8f39e838e5d7bd7435546</code> (strkey-decoded <code>CB0W0LFS...MMF6</code>)	✓ Confirmed
Endpoint ID in every transaction	<code>30600</code> (Stellar mainnet)	✓ Confirmed
Required DVNs, send and receive config	LayerZero Labs, Canary, USDT0 (addresses resolved per chain)	✓ Confirmed
Receive confirmations (<code>Stellar → <chain></code>)	<code>320</code> on all 21 chains	✓ Confirmed
Enforced options (<code><chain> → Stellar</code>)	type 1: <code>lzReceive</code> gas 500,000; type 2: <code>lzReceive</code> gas 500,000 + <code>compose[0]</code> gas 500,000 (byte-identical options on all 21 chains)	✓ Confirmed
Send/receive libraries	the chain's current on-chain default, written back explicitly	✓ Confirmed

Check	Expected	Result
Executor and <code>maxMessageBytes</code> (send config)	current on-chain values, written back explicitly	✓ Confirmed
Transaction target	the registered USDT0 OFT (OApp) on that chain	✓ Confirmed
OApp owner/delegate	the expected USDT0 owner multisig on that chain	✓ Confirmed

The libraries and executor values were checked against both LayerZero's published deployment addresses and the live on-chain defaults for endpoint ID 30600 (`defaultSendLibrary`, `defaultReceiveLibrary`, `getExecutorConfig`). Every transaction target appears in USDT0's published contract-deployments page, and each target's `endpoint()` points back at the canonical LayerZero endpoint of its chain. On each of the 21 chains the OApp owner and the endpoint delegate are one and the same address, matching Everdawn's documented Safe for that chain. One observation: the Conflux owner Safe (3-of-5) includes the deployer EOA (`0x1a6362AD...`) as a signer in place of one of the five keys used elsewhere (`0x00F6D2b4...`); this deviates from the signer set used on the other chains and may be worth confirming as intentional.

Send confirmations per chain (<chain> → Stellar)

Chain	Expected confirmations	Result
Arbitrum	<code>4000</code> (pathway-specific override; other Arbitrum pathways stay at the default <code>60</code>)	✓ Confirmed
Bera	<code>60</code>	✓ Confirmed
Conflux	<code>2800</code>	✓ Confirmed
Ethereum	<code>65</code>	✓ Confirmed
Flare	<code>500</code>	✓ Confirmed
Hedera	<code>60</code>	✓ Confirmed
Hyperliquid	<code>43200</code>	✓ Confirmed
Ink	<code>3000</code>	✓ Confirmed
Mantle	<code>2000</code>	✓ Confirmed

Chain	Expected confirmations	Result
MegaETH	5400	✓ Confirmed
Monad	3600	✓ Confirmed
Morph	1000	✓ Confirmed
Optimism	1000	✓ Confirmed
Plasma	1800	✓ Confirmed
Polygon	32	✓ Confirmed
Rootstock	120	✓ Confirmed
Sei	2000	✓ Confirmed
Stable	3600	✓ Confirmed
Tempo	3000	✓ Confirmed
Unichain	1600	✓ Confirmed
XLayer	9000	✓ Confirmed

Execution status (point-in-time)

Check	Expected	Result
Remote-side wiring transactions	not yet executed; <code>peers(30600)</code> unset on all 21 chains	✓ Confirmed
Unwired-pathway behavior	default DVN for the pathway is <code>LZDeadDVN</code> , so no message can transit until the wiring lands (fail-closed)	✓ Confirmed
Stellar OFT peers (Soroban RPC)	peers set for all 21 remote endpoint IDs and matching the corresponding USDT0 OFT addresses	✓ Confirmed

Stellar-side configuration (read on-chain via Soroban)

The Stellar-side configuration beyond peers was not part of the shared material, so it was read directly from the deployed contracts. All values are explicitly set (not inherited defaults).

Check	Expected / Observed	Result
OFT <code>owner</code> / <code>endpoint</code>	OneSig <code>CBCZ5CET...QKR6</code> / canonical LayerZero endpoint <code>CCQLLRE5...TZGK</code>	✓ Confirmed
Send and receive library (all 21 pathways)	<code>CCV4HEII...GOBA</code> , LayerZero's canonical Stellar message library (one contract serves both directions), explicitly set	✓ Confirmed
Send ULN config (<code>Stellar → <chain></code>)	confirmations <code>320</code> , required DVNs = LayerZero Labs, Canary, USDT0 (Stellar deployments), no optional DVNs — mirrors the remote receive config	✓ Confirmed
Receive ULN config (<code><chain> → Stellar</code>)	per-chain confirmations equal to the send-confirmations table above (e.g. Arbitrum <code>4000</code>), same three required DVNs — mirrors the remote send config	✓ Confirmed
Executor config	canonical LayerZero Stellar executor, <code>max_message_size 10000</code> on all pathways	✓ Confirmed
Enforced options (<code>Stellar → <chain></code>)	observed: <code>lzReceive</code> gas 80k / 100k+100k compose baseline, with higher per-chain values on Conflux, Polygon, Sei (150k), MegaETH (200k), Monad (240k), Tempo (550k)	✓ Read (no client-stated expectation to compare against)

Both halves of every pathway are mutually consistent: the Stellar receive parameters mirror each remote chain's proposed send parameters, and vice versa. The Stellar-side enforced-option gas values were not part of the client's shared configuration; they are reported as observed and may be worth a one-line confirmation from the client.

Trust Assumptions and Privileged Roles

- **OneSig** is the root owner of both the OFT and the SAC Manager. The owner grants and revokes all RBAC roles and holds upgrade authority, so it is the ultimate authority over both contracts once the issuer is locked. Its 3-of-5 signer set, threshold, and executor requirement are all mutable through OneSig itself, so the values above are a snapshot at the time of review.
- The asset issuer ([GATISXX6...](#)) was the SAC's original admin and a classic asset can mint from its own account. This is neutralized only by the master-weight-0 lock.
- **OneSig code is not reviewed here.** This report verifies only that the deployed OneSig WASM rebuilds from source and that it is the owner of the USDT0 contracts.
- **LayerZero pathway wiring** is covered as a review of the *proposed* remote-side calldata and of point-in-time on-chain state (see "LayerZero Pathway Wiring"). At the time of review (2026-08-04) the remote-side transactions had not been executed, and all wiring values (peers, libraries, DVN/executor configuration, enforced options) remain mutable by the OApp owner/delegate on each chain and by OneSig on Stellar. The Stellar-side configuration was not part of the shared material and was instead read directly from the deployed contracts; it is a point-in-time snapshot on the same terms.

Conclusion

OpenZeppelin performed a deployment verification of the USDT0 contracts on the Stellar network: the OFT, the SAC Manager, the OneSig multisig owner, and the network built-in USDT0 SAC. The verification covered reproducible-build equivalence for the three custom contracts, the identity of the SAC, and the ownership chain that makes the OneSig multisig the root owner of the deployed contracts.

All three custom contracts rebuilt byte-for-byte from their published source archives via the SEP-58 verifiable-build process, and each archive's SHA-256 matched the `source_sha256` recorded in the corresponding on-chain binary. The USDT0 SAC re-derived to its expected address and was confirmed to run network built-in code. The ownership chain resolved as intended: the SAC admin is the SAC Manager, and both the SAC Manager and the OFT are owned by the OneSig multisig (3-of-5). The asset issuer has been locked (master key weight `0`, no other signer).

OneSig's contract logic and the LayerZero pathway wiring were outside the scope of this review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.